

Math Library

User Manual

Abstract

This document describes the math library of macros and routines. This library can be used for MLX16 and MLX16x8 etc based products.

History records

Date	Revision	Comments
Sep 2007	1.0	Stand-alone version
1 Oct, 2007	1.5	Outlined together with MelexCM S/W Platform rev 1.5
12 Dec, 2008	2.0	Added mulhi32_32by16 routines Added mul24_16by8 routines Added mul16_8by8 routines Added isqrt routines Added ilog2, iexp2 routines Added trigonometric routines Added vector multiplication (DSP) routines Added mul16hi_16by8 routines
	2.1	Added div8_8by8 routines Added div8hi_8by8 routines
		Added _javg routine
		Added mulQ15_Q15byQ15 routine
		Added bitrev16, crc8 routines Added configuration and customization section
		Added DSP lib routines vecsumI32_I16 vecsumI32_I32 vecsumI48_I32 norm2U32_I16byI16 norm2U48_I16byI16 norm2vectorU32_I16byI16 norm2vectorU48_I16byI16 dotproductI32_I16byI16 dotproductI48_I16byI16 norm1vectorU32_I16 norm1vectorU32_I32 norm1vectorU48_I32
		Added CRC library routines Added DSP filter library routines
		Added DSP lib routines vecsumU8_U8 vecsumI8_I8 vecsumU16_U8 vecsumI16_I8 vecsumU16_U16 vecsumI16_I16
12 Dec, 2008	2.2	Replaced non-standard int types by standard one from <stdint.h>

Table of Contents

1. ABOUT THIS DOCUMENT	5
1.1. PURPOSE AND SCOPE	5
1.2. DOCUMENT CONVENTIONS	5
1.3. GLOSSARY OF TERMS / ABBREVIATIONS	5
1.4. REFERENCES	5
2. OVERVIEW.....	6
2.1. LIBRARY STRUCTURE	6
2.1.1. DSP module.....	6
2.1.2. Filter module	6
2.1.3. CRC module	7
2.2. LIBRARY USE	7
2.3. LIBRARY INSTALLATION.....	7
2.4. LIBRARY REQUIREMENTS	8
3. MULTIPLICATION FUNCTIONS.....	8
3.1. MULTIPLICATION OF 16-BIT BY 16-BIT INTEGER NUMBERS WITH 32-BIT RESULT	8
3.2. MULTIPLICATION OF 16-BIT BY 16-BIT INTEGER NUMBERS WITH 16-BIT RESULT	8
3.3. MULTIPLICATION OF 32-BIT BY 16-BIT INTEGER NUMBERS WITH 32-BIT RESULT	8
3.4. MULTIPLICATION OF 32-BIT BY 16-BIT INTEGER NUMBERS WITH 32-BIT MSB RESULT.....	9
3.5. MULTIPLICATION OF 16-BIT BY 8-BIT INTEGER NUMBERS WITH 24-BIT RESULT	9
3.6. MULTIPLICATION OF 16-BIT BY 8-BIT INTEGER NUMBERS WITH 16-BIT RESULT	9
3.7. MULTIPLICATION OF 8-BIT BY 8-BIT INTEGER NUMBERS WITH 16-BIT RESULT	10
3.8. MULTIPLICATION OF 16-BIT SIGNED FRACTION BY 16-BIT SIGNED FRACTION NUMBERS WITH 16-BIT SIGNED FRACTION RESULT	10
4. DIVISION FUNCTIONS	11
4.1. DIVISION OF 32-BIT BY 16-BIT INTEGER NUMBERS WITH 16-BIT RESULT.....	11
4.2. DIVISION OF 32-BIT BY 16-BIT INTEGER NUMBERS WITH 32-BIT RESULT.....	11
4.3. DIVISION OF 16-BIT BY 16-BIT INTEGER NUMBERS WITH 16-BIT RESULT.....	11
4.4. DIVISION OF 8-BIT BY 8-BIT INTEGER NUMBERS WITH 8-BIT RESULT	12
4.5. DIVISION OF 8-BIT BY 8-BIT INTEGER NUMBERS WITH 8-BIT RESULT WITH DIVIDEND SHIFTED LEFT BY 8 BITS	12
5. EXPONENTIATION FUNCTIONS.....	13
5.1. INTEGER SQUARE ROOT	13
5.2. INTEGER LOGARITHM WITH BASE 2	13
5.3. INTEGER EXPONENT (POWER) WITH BASE 2.....	13
6. DIGITAL SIGNAL PROCESSING FUNCTIONS	14
6.1. AVERAGE OF TWO 16-BIT INTEGERS	14
6.2. SUM OF VECTORS WITH 8-BIT INTEGER NUMBERS WITH 8-BIT RESULT	14
6.3. SUM OF VECTORS WITH 8-BIT INTEGER NUMBERS WITH 16-BIT RESULT	15
6.4. SUM OF VECTORS WITH 16-BIT INTEGER NUMBERS WITH 16-BIT RESULT	15
6.5. SUM OF VECTORS WITH 16-BIT INTEGER NUMBERS WITH 32-BIT RESULT	16
6.6. SUM OF VECTORS WITH 32-BIT INTEGER NUMBERS WITH 32/48-BIT RESULT	17
6.7. NORM-1 OF VECTORS WITH 16-BIT INTEGER ELEMENTS WITH 32/48-BIT RESULT	17
6.8. NORM OF VECTORS WITH 16-BIT INTEGER ELEMENTS WITH 32/48-BIT RESULT.....	18
6.9. SCALAR (DOT, INNER) PRODUCT OF VECTORS WITH 16-BIT INTEGER NUMBERS WITH 32/48-BIT RESULT.....	19
7. DSP FILTER FUNCTIONS	20
7.1. FINITE IMPULSE RESPONSE FILTER FUNCTIONS	20
7.2. (FIRST ORDER) INFINITE IMPULSE RESPONSE FILTER FUNCTIONS	21
7.3. LOW PASS FILTER FUNCTIONS	21

8. TRIGONOMETRIC FUNCTIONS.....	22
8.1. SINE	23
8.2. COSINE	24
8.3. TANGENT.....	24
8.4. INVERSE TANGENT	25
9. OTHER FUNCTIONS	25
9.1. RAND32	25
9.2. BITREV	26
9.3. CRC	26
10. TRIGONOMETRIC FUNCTION APPROXIMATIONS.....	28
11. LIBRARY CUSTOMIZATION AND CONFIGURATION.....	29
11.1. C SIMULATIONS.....	29
11.2. TRIGONOMETRIC FUNCTION APPROXIMATIONS	29
11.3. CRC FUNCTIONS WITH NON-STANDARD POLYNOMIALS (LIBMATH.A)	29
11.4. CRC FUNCTIONS WITH NON-STANDARD POLYNOMIALS (LIBCRC.A).....	30
11.4.1. <i>Other polynomials</i>	30
11.4.2. <i>Reflected variants</i>	30
11.4.3. <i>256 entry table based implementation</i>	31
11.4.4. <i>16 entry table based implementation</i>	32
11.4.5. <i>Two 16 entry table based implementation</i>	33
11.5. (LOW PASS) FILTER FUNCTIONS WITH OTHER COEFFICIENTS (LIBFILTER.A).....	34

1. About this document

1.1. Purpose and scope

This document describes a software library with standard mathematical functions (further the library). The target audience of this document is software engineers who design firmware for MLX16 based systems.

1.2. Document conventions

Italic text: non-English words (e.g. *stdint.h*).

1.3. Glossary of Terms / Abbreviations

Abbreviation	Description
ALU	Arithmetic Logic Unit
CRC	Cyclic Redundancy Check
DSP	Digital signal processing
LSB	Least significant bit
MSB	Most significant bit

Term	Description
MLX16	16 bit micro-processor core from Melexis IP
MLX16x8	16 bit micro-processor with MAC coprocessor core from Melexis IP

1.4. References

The following documents are referred to in this document:

- [1] MISRA-C:2004 – Guidelines for the use of the C language in critical systems

2. Overview

The library is designed to ensure efficient calculation of arithmetical and other mathematical primitive functions which are commonly used in projects.

The software math library is available as standalone package or as a part of the SW platform.

The software math library consists of 4 modules:

- Math library – basic/core module extending the C library with arithmetical and other mathematical functions
- DSP module – library with digital signal processing functions on vectors
- Filter module – library containing (digital signal processing) filter functions
- CRC module – an extendable library of CRC functions of different sizes, polynomials and other variants

2.1. Library structure

The basic library is comprised of two header files and one lib file

`/include/mathlib.h`
`/lib/libmath.a`

The source code can be found in the following folder.

Standalone package:

`/libsrc/`

SW platform package:

`/libsrc/math/`

2.1.1. DSP module

The DSP library is comprised of one header file and one lib file:

`/include/dsp.h`
`/lib/libdsp.a`

The source code can be found in the following folder.

`/libsrc/dsp`

This module may depend on functions in the (basic) math library module.

2.1.2. Filter module

The filter library is comprised of one header file and one lib file:

`/include/filter.h`
`/lib/libfilter.a`

The source code can be found in the following folder.

/libsrc/filter

This module may depend on functions in the (basic) math library module.
This module may depend on functions in the dsp module. (future)

2.1.3. CRC module

The CRC library is comprised of one header file and one lib file:

/include/crc.h
/lib/libcrc.a

The source code can be found in the following folder.

/libsrc/crc

This module can be used stand-alone and has no dependency on the (basic) math library module.

Note that the (basic) math library contains also some elementary CRC routines.

2.2. Library use

To use the (basic) library within a project, it is sufficient to only include *mathlib.h* header in the source files, and to link with the *libmath.a*. Please see */test* folder for stand-alone package and */projects/Validation/MATHLIB* for SW platform as example of library usage.

To use the other library modules within a project, it is sufficient to include the respective *<module.h>* header in the source files, and to also link with the *lib<module>.a*.

The *mathlib.h* header defines the function prototypes of all available functions in the math library.
The *dsp.h* header defines the function prototypes of all available functions in the DSP library.
The *filter.h* header defines the function prototypes of all available functions in the filter library.
The *crc.h* header defines the function prototypes of all available functions in the CRC library.

Most of the functions of the basic math library and of the DSP module are optimized in assembler. The C-code is given for illustration purpose only. There are different functions for different arguments types – signed and unsigned.

Most of the functions of the other modules are implemented in C albeit specifically designed for efficient execution on the MLX16 CPU targets, using the special instructions and features of these CPU targets as applicable.

Functions that have optimized assembly versions, also has a C-code version. Use the LIB_SIMULATION variable in the */libsrc/math/Makefile* file to switch between C-code and optimized assembly code, (See Section 11.1)

2.3. Library installation

Depending on target CPU, makefile variable INSTSET shall be set to *-mlx16-x8*, *-mlx16-8* etc

To control the installation of the other library modules, the makefile variable SUBMODULE shall include the module name.

After copying the library to the appropriate place it should be compiled and installed. The following targets are defined in the Makefile:

- **all** – compiles the library but does not copy archive (library) file to the /lib directory
- **install** – executes 'all' and copies archive (library) file to the /lib directory
- **clean** – clean-up of the source code from the object files. Note: does not remove anything from the /lib directory.

See also Section 11 on configuring, customizing and extending the library.

2.4. Library requirements

| The math library release versions 2.x requires MLX16-GCC release 1.8 or later.

3. Multiplication functions

3.1. Multiplication of 16-bit by 16-bit integer numbers with 32-bit result

```
int32_t mulI32_I16byI16(int16_t x, int16_t y)
int32_t mulI32_I16byU16(int16_t x, uint16_t y)
uint32_t mulU32_U16byU16(uint16_t x, uint16_t y)
```

These functions calculate the 32-bit integer result of the multiplication of two 16-bit integer arguments.

```
int32_t mulI32_I16byI16(int16_t x, int16_t y)
{
    return (int32_t)x*y;
}
```

C-code 1

3.2. Multiplication of 16-bit by 16-bit integer numbers with 16-bit result

```
int16_t mulI16_I16byI16(int16_t x, int16_t y)
int16_t mulI16_I16byU16(int16_t x, uint16_t y)
uint16_t mulU16_U16byU16(uint16_t x, uint16_t y)
```

These functions calculate the 16 MSBs, i.e. 16 bit high part, of the result of the multiplication of 2 16-bit integer arguments.

```
int16_t mulI16_I16byI16(int16_t x, int16_t y)
{
    int32_t acc;
    acc = (int32_t)x*y;
    return (int16_t)(acc >> 16);
}
```

C-code 2

3.3. Multiplication of 32-bit by 16-bit integer numbers with 32-bit result

```
int32_t mulI32_I32byI16(int32_t x, int16_t y)
int32_t mulI32_I32byU16(int32_t x, uint16_t y)
uint32_t mulU32_U32byU16(uint32_t x, uint16_t y)
```

These functions calculate the 32 LSBs, i.e. 32-bit low part, of the result of the multiplication of a 32-bit integer and a 16-bit integer argument. The multiplication result of the full range 32-bit and 16-bit arguments is a 48-bit integer number but sometimes it is known that arguments are limited in a smaller range and the result lies in the 32-bit range (e.g. x: 21-bit and y: 9-bit).


```
int32_t mulI32_I32byI16(int32_t x, int16_t y)
{
    int32_t acc;
    acc = (int32_t)y * (x >> 16);
    acc <<= 16;
    acc += (int32_t)y * ((uint16_t)x);
    return acc;
}
```

C-code 3

3.4. Multiplication of 32-bit by 16-bit integer numbers with 32-bit MSB result

```
int32_t mulI32hi_I32byI16(int32_t x, int16_t y)
int32_t mulI32hi_I32byU16(int32_t x, uint16_t y)
uint32_t mulU32hi_U32byU16(uint32_t x, uint16_t y)
```

These functions calculate the 32 MSBs, i.e. 32-bit high part, of the result of the multiplication of a 32-bit integer and a 16-bit integer argument. The multiplication result of the full range 32-bit and 16-bit arguments is a 48-bit integer number, for which these functions then return the 32 MSBs of this 48-bit integer number. The 16 LSBs of the 48-bit integer number are truncated, also for negative values.

```
int32_t mulI32_I32byI16(int32_t x, int16_t y)
{
    int64_t acc;
    acc = (int64_t)x*y;
    return (int32_t)(acc >> 16);
}
```

C-code 4

3.5. Multiplication of 16-bit by 8-bit integer numbers with 24-bit result

```
int32_t mulI24_I16byI8(int16_t x, int8_t y)
int32_t mulI24_I16byU8(int16_t x, uint8_t y)
uint32_t mulU24_U16byU8(uint16_t x, uint8_t y)
```

These functions calculate the 24-bit integer result of the multiplication of a 16-bit integer with a 8-bit integer. The result is returned as a 32-bit value.

```
int32_t mulI24_I16byI8(int16_t x, int8_t y)
{
    int32_t acc;
    acc = (int32_t)x*y;
    return acc;
}
```

C-code 5

3.6. Multiplication of 16-bit by 8-bit integer numbers with 16-bit result

```
int32_t mulI16hi_I16byI8(int16_t x, int8_t y)
int32_t mulI16hi_I16byU8(int16_t x, uint8_t y)
uint32_t mulU16hi_U16byU8(uint16_t x, uint8_t y)
```

These functions calculate the 16 MSBs, i.e. 16-bit high part, of the result of the multiplication of a 16-bit integer and a 8-bit integer argument. The multiplication result of the full range 16-bit and 8-bit arguments is a 24-bit integer number, for which these functions then return the 16 MSBs of this 24-bit integer number. The 8 LSBs of the 24-bit integer number are truncated, also for negative values.

```
int16_t mulI16hi_I16byI8(int16_t x, int8_t y)
{
    int32_t acc;
    acc = (int32_t)x*y;
    return (int16_t) (acc >> 8);
}
```

C-code 6

3.7. Multiplication of 8-bit by 8-bit integer numbers with 16-bit result

```
int16_t mulI16_I8byI8(int8_t x, int8_t y)
int16_t mulI16_I8byU8(int8_t x, uint8_t y)
uint16_t mulU16_U8byU8(uint8_t x, uint8_t y)
```

These functions calculate the 16-bit integer result of the multiplication of two 8-bit integers.

```
int16_t mulI16_I8byI8(int8_t x, int8_t y)
{
    int16_t acc;
    acc = (int16_t)x*y;
    return acc;
}
```

C-code 7

3.8. Multiplication of 16-bit signed fraction by 16-bit signed fraction numbers with 16-bit signed fraction result

```
int16_t mulQ15_Q15byQ15(int16_t x, int16_t y)
```

This functions calculate the 16 bit signed fraction result, i.e. Q15 format, of the multiplication of 2 16-bit signed fraction [-1 .. 1) arguments, i.e. Q15 format.

Q15 format is 16 bit integer representation format of a value in range [-1 .. 1) with resolution of 2^{-15} .

```
int16_t mulQ15_Q15byQ15(int16_t x, int16_t y)
{
    int32_t acc;
    acc = (int32_t)x*y;
    return (int16_t) (acc >> 15);
}
```

C-code 8

```
static __inline__ int16_t _q15(int32_t a)
```

This function converts a 30 bit signed fraction value, i.e. Q30 format, to a 15 bit signed fraction value, i.e. Q15 format.

```
int16_t _q15(int32_t a){  
    return (int16_t) (a >> 15);  
}
```

C-code 9

4. Division functions

4.1. Division of 32-bit by 16-bit integer numbers with 16-bit result

```
uint16_t divU16_U32byU16(uint32_t a, uint16_t b)  
int16_t  divI16_I32byI16(int32_t a, int16_t b)  
int16_t  divI16_I32byU16(int32_t a, uint16_t b)
```

These functions calculate the 16-bit integer result of the division 32-bit dividend by 16-bit divisor.

The result should fit 16-bit. Otherwise incorrect calculation occurs due to overflow. There is no internal check for overflow inside the function.

The result is undefined for a zero divisor. There is no internal check on valid divisor, i.e. non-zero, inside the function.

```
int16_t divI16_I32byI16(int32_t a, int16_t b)  
{  
    return (int16_t) (a / b);  
}
```

C-code 10

4.2. Division of 32-bit by 16-bit integer numbers with 32-bit result

```
uint32_t divU32_U32byU16(uint32_t a, uint16_t b)  
int32_t  divI32_I32byI16(int32_t a, int16_t b)  
int32_t  divI32_I32byU16(int32_t a, uint16_t b)
```

These functions calculate the 32-bit integer result of the division 32-bit dividend by 16-bit divisor.

There is no limitation for the input arguments.

The result is undefined for a zero divisor. There is no internal check on valid divisor, i.e. non-zero, inside the function.

```
int32_t divI32_I32byI16(int32_t a, int16_t b)  
{  
    return a / b;  
}
```

C-code 11

4.3. Division of 16-bit by 16-bit integer numbers with 16-bit result

```
uint16_t divU16_U16byU16(uint16_t a, uint16_t b)  
int16_t  divI16_I16byI16(int16_t a, int16_t b)
```

```
int16_t divI16_I16byU16(int16_t a, uint16_t b)
```

These functions calculate the 16-bit integer result of the division 16-bit dividend by 16-bit divisor. The dividend shifted left before division by 16.

The dividend should be less than divisor. Otherwise incorrect calculation occurs due to overflow. There is no internal check for overflow inside the function.

The result is undefined for a zero divisor. There is no internal check on valid divisor, i.e. non-zero, inside the function.

```
int16_t divI16_I16byI16(int16_t a, int16_t b)
{
    int32_t acc = ((int32_t)a) << 16;
    return (uint16_t)(acc / b);
}
```

C-code 12

4.4. Division of 8-bit by 8-bit integer numbers with 8-bit result

```
uint8_t divU8_U8byU8(uint8_t a, uint8_t b)
int8_t divI8_I8byI8(int8_t a, int8_t b)
int8_t divI8_I8byU8(int8_t a, uint8_t b)
```

These functions calculate the 8-bit integer result of the division 8-bit dividend by 8-bit divisor.

The result is undefined for a zero divisor. There is no internal check on valid divisor, i.e. non-zero, inside the function.

```
int8_t divI8_I8byI8(int8_t a, int8_t b)
{
    return (int8_t)((int16_t) a / (int16_t) b);
}
```

C-code 13

4.5. Division of 8-bit by 8-bit integer numbers with 8-bit result with dividend shifted left by 8 bits

```
uint8_t divU8hi_U8byU8(uint8_t a, uint8_t b)
int8_t divI8hi_I8byI8(int8_t a, int8_t b)
int8_t divI8hi_I8byU8(int8_t a, uint8_t b)
```

These functions calculate the 8-bit integer result of the division 8-bit dividend by 8-bit divisor. The dividend shifted left before division by 8.

The dividend should be less than divisor. Otherwise incorrect calculation occurs due to overflow. There is no internal check for overflow inside the function.

The result is undefined for a zero divisor. There is no internal check on valid divisor, i.e. non-zero, inside the function.

```
int8_t divI8hi_I8byI8(int8_t a, int8_t b)
{
    return (int8_t) ((int16_t) n * 256 / (int16_t) d);
}
```

C-code 14

5. Exponentiation functions

The functions described in this section make use of the *fsb* and *sfb* instructions for the MLX16x8 and MLX16-8 CPU targets.

5.1. Integer square root

```
uint16_t isqrt16 (uint16_t x)
uint16_t isqrt32 (uint32_t x)
```

These functions calculate the integer part of the square root of the input argument *x*. I.e. they return the greatest integer less than or equal to the exact square root.

5.2. Integer logarithm with base 2

```
uint16_t ilog2_U16 (uint16_t x)
uint16_t ilog2_U32 (uint32_t x)
```

These functions calculate the integer part of the logarithm with base 2 of the input argument $x > 0$. I.e. they return the greatest integer n for which $2^n \leq x$.

! The functions return 0xFFFF for an input argument with value 0.

```
static __inline__ int16_t _fsb (uint16_t a)
static __inline__ int16_t _fsb32 (uint32_t a)
```

These supporting functions are used by the primary API function and perform the actual calculation (for non-zero values).

5.3. Integer exponent (power) with base 2

```
uint16_t iexp2_U16 (uint16_t x)
uint32_t iexp2_U32 (uint16_t x)
```

These functions calculate 2^x .

The functions are only specified for values within the specified range, i.e.

- $0 \leq x < 16$, for *iexp2_U16*
- $0 \leq x < 32$, for *iexp2_U32*

```
static __inline__ uint16_t _sfb (uint16_t a)
static __inline__ uint32_t _sfb32 (uint16_t a)
```

These supporting functions are used by the primary API functions and perform the actual calculation.

! The inline expansion may result in relatively large code for the MLX16 CPU target.

6. Digital signal processing functions

The functions described in this section make use of the macu/s instructions for the MLX16x8 CPU target which are not used by the MLX16 GCC version 1.x.

6.1. Average of two 16-bit integers

```
static __inline__ uint16_t _uavg (uint16_t a, uint16_t b)
static __inline__  int16_t _iavg (int16_t a, int16_t b)
```

These functions calculate the average of two 16-bit integers for all values. The functions take care of overflow, while not relying on an intermediate 32 bit value.

```
uint16_t averageU16 (uint16_t a, uint16_t b)
{
    uint32_t sum;

    sum = (uint32_t) a + b;
    sum /= 2;

    return (uint16_t) sum;
}

int16_t averageI16 (int16_t a, int16_t b)
{
    int32_t sum;

    sum = (int32_t) a + b;
    sum /= 2;

    return (int16_t) sum;
}
```

C-code 15

6.2. Sum of vectors with 8-bit integer numbers with 8-bit result

```
uint8_t vecsumU8_U8 (const uint8_t *a, uint16_t n) /* dsp */
int8_t  vecsumI8_I8 (const int8_t *a, uint16_t n) /* dsp */
```

These functions calculate the sum of the elements of the specified vector of 8-bit integer values and of length $n \geq 0$, i.e.

$$\sum_{0 \leq i < n} a[i]$$

The result is returned as a 8-bit integer.

Length n is not constrained for any CPU target

All intermediate values are stored as 8 bit values. There are no internal checks for overflow inside the function.
There is no internal check on valid length n inside the function.

```
uint8_t vecsumU8_U8(uint8_t *a, uint16_t n)
{
    int i;
    uint8_t sum = 0;

    for (i = 0; i < n; i++) {
        sum += a[i];
    }

    return sum;
}
```

C-code 16

6.3. Sum of vectors with 8-bit integer numbers with 16-bit result

```
uint16_t vecsumU16_U8 (const uint8_t *a, uint16_t n) /* dsp */
int16_t vecsumI16_I8 (const int8_t *a, uint16_t n) /* dsp */
```

These functions calculate the sum of the elements of the specified vector of 8-bit integer values and of length $n \geq 0$, i.e.

$$\sum_{0 \leq i < n} a[i]$$

The result is returned as a 16-bit integer.

Length n is not constrained for any CPU target

All intermediate values are stored as 16 bit values. There are no internal checks for overflow inside the function.

There is no internal check on valid length n inside the function.

```
uint16_t vecsumU16_U8(uint8_t *a, uint16_t n)
{
    int i;
    uint16_t sum = 0;

    for (i = 0; i < n; i++) {
        sum += a[i];
    }

    return sum;
}
```

C-code 17

6.4. Sum of vectors with 16-bit integer numbers with 16-bit result

```
uint16_t vecsumU16_U16 (const uint16_t *a, uint16_t n) /* dsp */
int16_t vecsumI16_I16 (const int16_t *a, uint16_t n) /* dsp */
```

These functions calculate the sum of the elements of the specified vector of 16-bit integer values and of length $n \geq 0$, i.e.

$$\sum_{0 \leq i < n} a[i]$$

The result is returned as a 16-bit integer.

Length n is not constrained for any CPU target

All intermediate values are stored as 16 bit values. There are no internal checks for overflow inside the function.

There is no internal check on valid length n inside the function.

```
uint16_t vecsumU16_U16(uint16_t *a, uint16_t n)
{
    int i;
    uint16_t sum = 0;

    for (i = 0; i < n; i++) {
        sum += a[i];
    }

    return sum;
}
```

C-code 18

6.5. Sum of vectors with 16-bit integer numbers with 32-bit result

```
uint32_t vecsumU32_U16 (const uint16_t *a, uint16_t n)
int32_t vecsumI32_I16 (const int16_t *a, uint16_t n) /* dsp */
```

These functions calculate the sum of the elements of the specified vector of 16-bit integer values and of length $n \geq 0$, i.e.

$$\sum_{0 \leq i < n} a[i]$$

The result is returned as a 32-bit integer.

Length n is constrained to $0 < n \leq 16$ for MLX16 and MLX16-8 CPU targets. Length is not constrained for the MLX16-x8 CPU target.

All intermediate values are stored as 32 bit values. There are no internal checks for overflow inside the function.

There is no internal check on valid length n inside the function.

```
uint32_t vecsumU32_U16(uint16_t *a, uint16_t n)
{
    int i;
    uint32_t sum = 0;

    for (i = 0; i < n; i++) {
        sum += a[i];
    }

    return sum;
}
```

C-code 19

6.6. Sum of vectors with 32-bit integer numbers with 32/48-bit result

```
uint32_t vecsumU32_U32 (const uint32_t *a, uint16_t n)
int32_t vecsumI32_I32 (const int32_t *a, uint16_t n) /* dsp */
uint32_t vecsumU48_U32 (const uint32_t *a, uint16_t n, uint16_t *msw)
int32_t vecsumI48_I32 (const int32_t *a, uint16_t n, int16_t *msw) /* dsp*/
```

These functions calculate the sum of the elements of the specified vector of 32-bit integer values and of length $n \geq 0$, i.e.

$$\sum_{0 \leq i < n} a[i]$$

The result is returned as a 32-bit integer. The 16 MSBs of the 48-bit result, i.e. 16 bit high part, is stored in the location pointed by *msw*.

Length n is constrained to $0 < n \leq 16$ for MLX16 and MLX16-8 CPU targets. Length is not constrained for the MLX16-x8 CPU target

For the functions `vecsumU32_U32` and `vecsumI32_I32`, all intermediate values are stored as 32 bit values, and there are no internal checks for overflow in these functions.¹
There is no internal check on valid length n inside the function.
There is no internal check on valid pointer *msw*, i.e. non-null, inside the function.

```
uint32_t vecsumU48_U32(const uint32_t *a, uint16_t n, uint16_t *msw)
{
    int i;
    uint64_t sum = 0;

    for (i = 0; i < n; i++) {
        sum += a[i];
    }

    *msw = (uint16_t) (sum >> 32);

    return (uint32_t) sum;
}
```

C-code 20

6.7. Norm-1 of vectors with 16-bit integer elements with 32/48-bit result

```
uint32_t norm1vectorU32_I16(const int16_t *a, uint16_t n) /* dsp */
uint32_t norm1vectorU32_I32(const int32_t *a, uint16_t n) /* dsp */
uint32_t norm1vectorU48_I32(const int32_t *a, uint16_t n, uint16_t *msw) /*
dsp */
```

These functions calculate the 1-norm (Manhattan norm) of the specified vector of 16-bit integer values and of length $n \geq 0$, i.e.

¹ For the functions `vecsumU48_U32` and `vecsumI48_I32`, there is no internal check on overflow of *msw* which could only be applicable for vector lengths $\geq 2^{16}$.

$$\sum_{0 \leq i < n} |a[i]|$$

The result is returned as a 32-bit integer. The 16 MSBs of the 48-bit result, i.e. 16 bit high part, is stored in the location pointed by *msw*.

Length n is constrained to $0 < n \leq 16$ for all CPU targets, except for `norm1vectorU32_I16` in case of MLX16-x8 CPU target.

Except for function `norm1vectorU48_I32`, all intermediate values are stored as 32 bit values and there are no internal checks for overflow.²
There is no internal check on valid length n inside the function.
There is no internal check on valid pointer *msw*, i.e. non-null, inside the function.

```
uint32_t norm1vectorU32_I16 (const int16_t *a, uint16_t n)
{
    unsigned int i;
    uint32_t sum = 0;

    for (i = 0; i < n; i++) {
        sum += abs(*a);
        a++;
    }
    return sum;
}
```

C-code 21

6.8. Norm of vectors with 16-bit integer elements with 32/48-bit result

```
uint32_t norm2U32_U16byU16 (uint16_t a, uint16_t b)
uint32_t norm2U48_U16byU16 (uint16_t a, uint16_t b, uint16_t *msw)
uint32_t norm2U32_I16byI16 ( int16_t a,  int16_t b) /* dsp */
uint32_t norm2U48_I16byI16 ( int16_t a,  int16_t b, uint16_t *msw) /* dsp, not
effective */
```

These functions calculate the square of the Euclidian norm of a vector with two 16-bit integer elements a and b , i.e. $a^2 + b^2$. The result is returned as a 32-bit integer.

```
uint32_t norm2vectorU32_U16byU16(const uint16_t *a, uint16_t n)
uint32_t norm2vectorU32_I16byI16(const int16_t *a, uint16_t n) /* dsp */
uint32_t norm2vectorU48_U16byU16(const uint16_t *a, uint16_t n, uint16_t *msw)
uint32_t norm2vectorU48_I16byI16(const int16_t *a, uint16_t n, uint16_t *msw)
/* dsp */
```

These functions calculate the square of the Euclidian norm of the specified vector of 16-bit integer values and of length $n \geq 0$, i.e.

² For the function `norm1vectorU48_I32`, there is no internal check on overflow of *msw* which could only be applicable for vector lengths $\geq 2^{16}$.

$$\sum_{0 \leq i < n} a[i]^2$$

The result is returned as a 32-bit integer. The 16 MSBs of the 48-bit result, i.e. 16 bit high part, is stored in the location pointed by *msw*.

For `norm2vectorU48_U16byU16` and `norm2U48_I16byI16`, the length *n* is constrained to $0 < n \leq 16$ for all CPU targets.

Except for function `norm2vectorU48_U16byU16` and `norm2vectorU48_I16byI16`, all intermediate values are stored as 32 bit values and there are no internal checks for overflow.³
There is no internal check on valid length *n* inside the function.
There is no internal check on valid pointer *msw*, i.e. non-null, inside the function.

```
uint32_t norm2vectorU32_U16byU16 (const uint16_t *a, uint16_t n)
{
    int i;
    uint32_t sum = 0;

    for (i = 0; i < n; i++) {
        sum += ((uint32_t) a[i]) * a[i];
    }

    return sum;
}
```

C-code 22

6.9. Scalar (dot, inner) product of vectors with 16-bit integer numbers with 32/48-bit result

```
uint32_t dotproductU32_U16byU16 (const uint16_t *a, const uint16_t *b,
uint16_t n)
    int32_t dotproductI32_I16byI16 (const int16_t *a, const int16_t *b,
uint16_t n) /* dsp */
uint32_t dotproductU48_U16byU16 (const uint16_t *a, const uint16_t *b,
uint16_t n, uint16_t *msw)
    int32_t dotproductI48_I16byI16 (const int16_t *a, const int16_t *b,
uint16_t n, int16_t *msw) /* dsp */
```

These functions calculate the scalar product of the specified vectors of 16-bit integer values and of length $n \geq 0$, i.e.

$$\sum_{0 \leq i < n} a[i] \times b[i]$$

The result is returned as a 32-bit integer. The 16 MSBs of the 48-bit result, i.e. 16 bit high part, is stored in the location pointed by *msw*.

³ For the functions `norm2vectorU48_U16byU16` and `norm2vectorU48_I16byI16`, there is no internal check on overflow of *msw* which could only be applicable for vector lengths $\geq 2^{16}$.

Length n is constrained to $0 < n \leq 16$ for all CPU targets.

Except for functions *dotproductU48_U16byU16* and *dotproductI48_I16byI16*, all intermediate values are stored as 32 bit values and there are no internal checks for overflow.

There is no internal check on valid length n inside the function.

There is no internal check on valid pointer *msw*, i.e. non-null, inside the function.

```
uint32_t dotproductU32_U16byU16 (const uint16_t *a, const uint16_t *b,
uint16_t n)
{
    int i;
    uint32_t sum = 0;

    for (i = 0; i < n; i++) {
        sum += ((uint32_t) a[i]) * b[i];
    }

    return sum;
}
```

C-code 23

7. DSP Filter functions

7.1. Finite impulse response filter functions

```
void    fir_init(uint16_t *p_in, uint16_t init_value, uint16_t n)
uint16_t fir11    (uint16_t in, uint16_t *p_in)
uint16_t fir121    (uint16_t in, uint16_t *p_in)
uint16_t fir1111    (uint16_t in, uint16_t *p_in)
uint16_t fir1331    (uint16_t in, uint16_t *p_in)
uint16_t fir12221    (uint16_t in, uint16_t *p_in)
```

The *firnn* functions return the output of a finite impulse response (FIR) filter with the specified coefficients on input *in* and filter delay line values *p_in*, and update the filter delay line *p_in*. The output is calculated exactly, without loss of precision, and the entire input range ($0 \leq in < 65536$) is supported.

Before use of the *firnn* functions, the filter delay line *p_in* should be initialized with the function *fir_init*.

```

fir11:  (in + p_in[0]) / 2; p_in[0] = in
fir121: (in + 2 * p_in[0] + p_in[1]) / 4; p_in[1] = p_in[0]; p_in[0] = in
fir1111: (in + p_in[0] + p_in[1] + p_in[2]) / 4; p_in[2] = p_in[1]; p_in[1] = p_in[0]; p_in[0] = in
fir1331: (in + 3 * p_in[0] + 3 * p_in[1] + p_in[2]) / 8; p_in[2] = p_in[1]; p_in[1] = p_in[0]; p_in[0] = in
fir12221: (in + 2 * p_in[0] + 2 * p_in[1] + 2 * p_in[2] + p_in[3]) / 8; p_in[3] = p_in[2]; p_in[2] = p_in[1]; p_in[1] = p_in[0]; p_in[0] = in
```

The function *fir_init* initializes the filter delay line *p_in* of length n with value *init_value*.

```
void    firI16_init (int16_t *p_in, int16_t init_value, uint16_t n)
int16_t firI11_I16    (int16_t in, int16_t *p_in)
int16_t firI121_I16    (int16_t in, int16_t *p_in)
int16_t firI1111_I16    (int16_t in, int16_t *p_in)
int16_t firI1331_I16    (int16_t in, int16_t *p_in)
int16_t firI12221_I16    (int16_t in, int16_t *p_in)
```

The *firnn_I16* functions return the output of a finite impulse response of filter with the specified

coefficients on input *in* and filter delay line values *p_in*, and update the filter delay line *p_in*. The output is calculated exactly, without loss of precision, and the entire input range ($-32768 \leq in < 32768$) is supported.

Before use of the *firnnn_16* functions, the filter delay line *p_in* should be initialized with the function *fir16_init*.

The function *fir16_init* initializes the filter delay line *p_in* of length *n* with value *init_value*.⁴

7.2. (First order) Infinite impulse response filter functions

```
void iir_init      (uint16_t *p_out, uint16_t init, uint16_t n)
void iirI16_init  ( int16_t *p_out,  int16_t init, uint16_t n)
void iirU32_init  (uint32_t *p_out, uint16_t init, uint16_t n)
void iirI32_init  ( int32_t *p_out,  int16_t init, uint16_t n)

uint16_t iir1     (uint16_t in, uint16_t *p_out, uint16_t a)
uint16_t iir1U32  (uint16_t in, uint32_t *p_out, uint16_t a)
int16_t iir1I16   ( int16_t in,  int16_t *p_out, int16_t a)
int16_t iirI32    ( int16_t in,  int32_t *p_out, int16_t a)
```

The *iir1* functions return the output of a first order infinite impulse response (IIR) filter with coefficient *a* on input *in* and filter delay line values *p_out*, and update the filter delay line *p_out*.

$$out[k] = (1 - a) * in + a * out[k - 1] = in + a * (out[k] - in); p_out[0] = out;$$

The coefficient *a* is a 16 bit unsigned fraction - Q0.16 format, representing a floating point value in the range of [0 .. 1) with resolution of 2^{-16} .⁵

For *iir1U32* and *iir1I32*, the filter delay line values *p_out* are 32 bit unsigned values with 16 fractional bits – Q16.16 format.

The calculations will not exercise any overflow and underflow, and the entire input range is supported ($0 \leq in < 65536$ for *iir1* and *iir1U32*, and $-32768 \leq in < 32768$ for *iir1I16* and *iir1I32*).

Before use of the *iir1* functions, the filter delay line *p_out* should be initialized with the respective *iir_init* function.

7.3. Low pass filter functions

```
void  lpf_init (uint16_t *p_in, uint16_t init_value, uint16_t n)
uint16_t lpf1  (uint16_t in, uint16_t *p_out)
uint16_t lpf2  (uint16_t in, uint16_t *p_out)
uint16_t lpf3  (uint16_t in, uint16_t *p_out)
uint16_t lpf4  (uint16_t in, uint16_t *p_out)
uint16_t lpf5  (uint16_t in, uint16_t *p_out)
uint16_t lpf6  (uint16_t in, uint16_t *p_out)
uint16_t lpf7  (uint16_t in, uint16_t *p_out)
uint16_t lpf8  (uint16_t in, uint16_t *p_out)
```

⁴ Note that upon return the array *p_in* will **not** be filled with *init_value* itself but with a modified value as to allow the full input range support and efficient implementation of the *firnnn_16* functions. I.e. these functions constitute a true abstraction API.

⁵ Classical filter theory specifies first order IIR as $out[k] = (1+a) * in - a * out[k-1]$. The BIBO stability condition is $|a| < 1$, and no overflow and underflow will be exercised for $a < 0$. In the math library, *a* is implicitly negated.

The *lpfn* functions return the output of a first order low pass filter on input *in* and filter delay line values *p_out*, and update the filter delay line *p_out*.

The output is calculated exactly, without loss of precision, only on half of the input range, i.e. ($0 \leq in < 32768$).

$$out[k] = \frac{(2^n - 1) * out[k - 1] + in}{2^n}; p_out[0] = out;$$

This is a special version of 1st order IIR filter with coefficient $a = -2^{-n}$.

Before use of the *lpfn* functions, the filter delay line *p_out* should be initialized with the function *lpf_init*.

```
void    lpf_init (uint16_t *p_in, uint16_t init_value, uint16_t n)
uint16_t lpf1_U16 (uint16_t in, uint16_t *p_out)
uint16_t lpf2_U16 (uint16_t in, uint16_t *p_out)
uint16_t lpf3_U16 (uint16_t in, uint16_t *p_out)
uint16_t lpf4_U16 (uint16_t in, uint16_t *p_out)
uint16_t lpf5_U16 (uint16_t in, uint16_t *p_out)
uint16_t lpf6_U16 (uint16_t in, uint16_t *p_out)
uint16_t lpf7_U16 (uint16_t in, uint16_t *p_out)
uint16_t lpf8_U16 (uint16_t in, uint16_t *p_out)
```

The *lpfn_U16* functions return the output of a first order low pass filter on input *in* and filter delay line values *p_out*, and update the filter delay line *p_out*.

The output is calculated exactly, without loss of precision, and the entire input range, i.e. ($0 \leq in < 65536$), is supported.

Before use of the *lpfn_U16* functions, the filter delay line *p_out* should be initialized with the function *lpf_init*.

```
void    lpf_init (uint16_t *p_in, uint16_t init_value, uint16_t n)
int16_t lpf1_I16 (int16_t in, int16_t *p_out)
int16_t lpf2_I16 (int16_t in, int16_t *p_out)
int16_t lpf3_I16 (int16_t in, int16_t *p_out)
int16_t lpf4_I16 (int16_t in, int16_t *p_out)
int16_t lpf5_I16 (int16_t in, int16_t *p_out)
int16_t lpf6_I16 (int16_t in, int16_t *p_out)
int16_t lpf7_I16 (int16_t in, int16_t *p_out)
int16_t lpf8_I16 (int16_t in, int16_t *p_out)
```

The *lpfn_I16* functions return the output of a first order low pass filter on input *in* and filter delay line values *p_out*, and update the filter delay line *p_out*.

The output is calculated exactly, without loss of precision, and the entire input range, i.e. ($-32768 \leq in < 32768$), is supported.

Before use of the *lpfn_I16* functions, the filter delay line *p_out* should be initialized with the function *lpf_init*.

See also Section 11.5 on configuring, customizing and extending the library.

8. Trigonometric functions

To represent angles and values of the trigonometric functions, the following number encoding schemes are used in the following:

- 16 bit signed fraction of 2π - representing a floating point value for an angle in the range of $[-\pi, \pi)$ with resolution of $2^{-16} * 2\pi$.
- 16 bit unsigned fraction of 2π - representing a floating point value for an angle in the range of $[0, 2\pi)$ with resolution of $2^{-16} * 2\pi$.
- 16 bit signed fraction – Q1.15 format, i.e. representing a floating point value in the range of $[-1 .. 1)$ with resolution of 2^{-15} .
- 16 bit unsigned fractions - Q0.16 format, representing a floating point value in the range of $[0 .. 1)$ with resolution of 2^{-16} .
- 32 bit signed value with 16 fractional bits – Q15.16 format, i.e. representing a floating point value in the range of $[-2^{15} .. 2^{15})$ with resolution of 2^{-16} .

Value (hex)	16 bit signed fraction of 2π (rad)	16 bit unsigned fraction of 2π (rad)	Q1.15	Q0.16
0	0	0	0	0
0x2000	$\pi / 4$	$\pi / 4$	0.25	0.125
0x4000	$\pi / 2$	$\pi / 2$	0.5	0.25
0x6000	$3\pi / 4$	$3\pi / 4$	0.75	0.375
0x7FFF	$\sim \pi$	$\pi - 2^{-16}/2\pi$	0.999969482421875 ($= 1 - 2^{-15}$)	0.4999847412109375 ($= 0.5 - 2^{-16}$)
0x8000	$-\pi$	π	-1	0.5
0xA000	$-3\pi / 4$	$5\pi / 4$	-0.75	0.675
0xC000	$-\pi / 2$	$3\pi / 2$	-0.5	0.75
0xE000	$-\pi / 4$	$7\pi / 4$	-0.25	0.875
0xFFFF	$-2^{-16} / 2\pi$	$\sim 2\pi$	-2^{-15}	$\sim 1 (1 - 2^{-16})$

8.1. Sine

```
int16_t sinU16 (uint16_t x)
int16_t sinI16 (int16_t x)
```

These functions calculate (an approximation of) $\sin(x)$ where x is an angle value represented as a 16 bit unsigned or signed fraction of 2π . The result is represented as a 16 bit signed fraction, i.e. Q15 format.

```
int16_t6 sin_helper (uint16_t x)
```

This supporting function calculates (an approximation of) $\sin(x)$ for angles in the first octant of the unit circle, i.e. for $0 \leq x < 0x2000$ (representing $\pi/4$). x is an angle value represented as a 16 bit unsigned or signed fraction of 2π . The result is represented as a 16 bit signed fraction, i.e. Q15 format.

This function is only defined for $0 \leq x < 0x2000$ (representing $\pi/4$). Otherwise incorrect approximation value is returned. There is no domain check inside the function.

This function is used by the primary trigonometric API functions (*sinU16*, *sinI16*, *cosU16*, *cosI16*, ..) as applicable.

⁶ This supporting function is not declared in mathlib.h. The return type and value range may change in a future release of the library.

8.2. Cosine

```
int16_t cosU16 (uint16_t x)
int16_t cosI16 (int16_t x)
```

These functions calculate (an approximation of) $\cos(x)$ where x is an angle value represented as a 16 bit unsigned or signed fraction of 2π . The result is represented as a 16 bit signed fraction, i.e. Q15 format.

```
int16_t7 cos_helper (uint16_t x)
```

This supporting function calculates (an approximation of) $\cos(x)$ for angles in the first octant of the unit circle, i.e. for $0 \leq x < 0x2000$ (representing $\pi/4$). x is an angle value represented as a 16 bit unsigned or signed fraction of 2π . The result is represented as a 16 bit signed fraction, i.e. Q15 format.

⁷ This function is only defined for $0 \leq x < 0x2000$ (representing $\pi/4$). Otherwise incorrect approximation value is returned. There is no domain check inside the function.

This function is used by the primary trigonometric API functions (*sinU16*, *sinI16*, *cosU16*, *cosI16*, ..) as applicable.

8.3. Tangent

```
int32_t tanU16 (uint16_t x)
int32_t tanI16 (int16_t x)
```

These functions calculate (an approximation of) $\tan(x)$ where x is an angle value represented as a 16 bit unsigned or signed fraction of 2π . The result is represented as a 32 bit signed value with 16 fractional bits, i.e. Q15.16 format.

Note: as for the singular values for angles of $(2n+1)\pi/2$:

- $\tan U16(0x4000) = -2147483647$, i.e. $-0x7FFF\ FFFF\ U(0x8000\ 0001)$
- $\tan U16(0xC000) = -2147483647$, i.e. $-0x7FFF\ FFFF\ U(0x8000\ 0001)$

```
int32_t8 tan_helper (uint16_t x)
```

This supporting function calculates (an approximation of) $\tan(x)$ for angles in the first quadrant of the unit circle, i.e. for $0 \leq x < 0x4000$ (representing $\pi/2$). x is an angle value represented as a 16 bit unsigned or signed fraction of 2π . The result is represented as a 32 bit signed value with 16 fractional bits, i.e. Q15.16 format.

⁸ This function is only defined for $0 \leq x < 0x4000$ (representing $\pi/2$). Otherwise incorrect approximation value is returned. There is no domain check inside the function.

This function is used by the primary trigonometric API functions (*tanU16*, *tanI16*, ..) as applicable.

These functions may call *sin_helper*, *cos_helper* and 16 and 32 bit division routines, and are computational expensive.

⁷ This supporting function is not declared in *mathlib.h*. The return type and value range may change in a future release of the library.

⁸ This supporting function is not declared in *mathlib.h*. The return type and value range may change in a future release of the library.

8.4. Inverse tangent

```
int16_t atan2U16 (uint16_t y, uint16_t x)
int16_t atan2I16 (int16_t y, int16_t x)
```

These functions calculate (an approximation of) arctan (y / x) where y and x are two 16 bit integer numbers. The result is an angle value represented as a 16 bit signed fraction of 2π , i.e. Q15 format.

Mathematically atan2 is defined as:

For $y \neq 0$:

$$a \tan 2(y, x) = \text{sign}(y) \cdot \begin{cases} \varphi & x > 0 \\ \pi / 2 & x = 0 \\ \pi - \varphi & x < 0 \end{cases}, \text{ where } \text{sign}(y) = \begin{cases} +1 & y > 0 \\ -1 & y < 0 \end{cases}$$

and

$$a \tan 2(0, x) = \begin{cases} 0 & x > 0 \\ \text{undefined} & x = 0 \\ \pi & x < 0 \end{cases}$$

atan2(0,0) is defined as 0 by the implementation.

```
int16_t9 atan2_helper (uint16_t y, uint16_t x);
```

This supporting function calculates (an approximation of) atan2 (y / x) for positive values y and x and $y \leq x$. The result is an angle value represented as a 16 bit signed fraction of 2π , i.e. Q15 format.

This function is only defined for $0 \leq y \leq x \leq 0xFFFF$. Otherwise internal overflow may occur. There is no internal check for this input constraint.

Due to the limited input domain, the return value is an angle in the first octant of the unit circle, i.e. $0 \leq \text{atan2_helper}(y, x) \leq 0x2000$ (representing $\pi/4$).
atan2_helper (0,0) is defined as 0.

This function is used by the primary trigonometric API functions (*atan2U16*, *atan2I16*, ..) as applicable.

9. Other functions

9.1. rand32

```
uint32_t rand32 (uint32_t seed)
```

The function calculates next pseudo-random value based on the seed value. The seed value is a previous calculated pseudo-random value.

⁹ This supporting function is not declared in mathlib.h. The return type and value range may change in a future release of the library.

```
uint32_t rand32(uint32_t seed)
{
    seed = seed * 69069 + 12345;
    return seed;
}
```

C-code 24

9.2. *bitrev*

```
uint8_t bitrev8(uint8_t x)
uint16_t bitrev16(uint16_t x)
```

The function calculates bit-reverse value.

```
uint8_t bitrev8(uint8_t x)
{
    x = ((x & 0xaa) >> 1) | ((x & 0x55) << 1);
    x = ((x & 0xcc) >> 2) | ((x & 0x33) << 2);
    x = (x >> 4) | (x << 4);

    return x;
}
```

C-code 25

9.3. *crc*

```
uint8_t crc8(uint8_t x, uint8_t crc)
```

The function calculates the standard 8-bit CRC based on the input parameter x and previous value of the CRC.

```
uint16_t crc16(uint8_t x, uint16_t crc)
```

The function calculates the 16-bit Xmodem (CCITT) CRC based on the input parameter x and previous value of the CRC.

```
uint32_t crc32(uint8_t x, uint32_t crc) /* crc */
```

The function calculates the 32-bit IEEE 802.3 CRC based on the input parameter x and previous value of the CRC.

```
uint16_t crc16(uint8_t c, uint16_t crc)
{
    uint16_t i;
    crc = crc ^ (((uint16_t) c) << 8);
    for (i = 0; i < 8; ++i)
    {
        if ((crc & 0x8000) != 0)
            crc = (crc << 1) ^ 0x1021;
        else
            crc = crc << 1;
    }
    return crc;
}
```

C-code 26

```
uint8_t  crc8_table256 (uint8_t x, uint8_t crc) /* crc */
uint16_t crc16_table256(uint8_t x, uint16_t crc) /* crc */
uint16_t crc32_table256(uint8_t x, uint32_t crc) /* crc */
```

These functions calculate the CRC based on the input parameter x and previous value of the CRC with a table-based CRC implementation using a table of 256 entries.

```
uint8_t  crc8_table16 (uint8_t x, uint8_t crc) /* crc */
uint16_t crc16_table16(uint8_t x, uint16_t crc) /* crc */
uint16_t crc32_table16(uint8_t x, uint32_t crc) /* crc */
```

These functions calculate the CRC based on the input parameter x and previous value of the CRC with a table-based CRC implementation using a (single) table of 16 entries.

```
uint8_t  crc8_tabletwo16 (uint8_t x, uint8_t crc) /* crc */
uint16_t crc16_tabletwo16(uint8_t x, uint16_t crc) /* crc */
uint16_t crc32_tabletwo16(uint8_t x, uint32_t crc) /* crc */
```

These functions calculate the CRC based on the input parameter x and previous value of the CRC with a table-based CRC implementation using two tables of 16 entries.

```
uint8_t  crc8_table256_data8 (uint8_t *data, uint16_t length, uint8_t init) /*
crc */
uint16_t crc16_table256_data8(uint8_t *data, uint16_t length, uint16_t init)
/* crc */
uint32_t crc32_table256_data8(uint8_t *data, uint16_t length, uint32_t init)
/* crc */

uint8_t  crc8_table16_data8 (uint8_t *data, uint16_t length, uint8_t init) /*
crc */
uint16_t crc16_table16_data8(uint8_t *data, uint16_t length, uint16_t init) /*
crc */
uint32_t crc32_table16_data8(uint8_t *data, uint16_t length, uint32_t init) /*
crc */

uint8_t  crc8_tabletwo16_data8 (uint8_t *data, uint16_t length, uint8_t init)
/* crc */
uint16_t crc16_tabletwo16_data8(uint8_t *data, uint16_t length, uint16_t init)
/* crc */
uint32_t crc32_tabletwo16_data8(uint8_t *data, uint16_t length, uint32_t init)
/* crc */
```

These functions calculate the CRC of *data* array of size *length* (bytes) using previous values of CRC *init*, using the respective CRC and table based implementation.

```
uint8_t  crc8_table256_data16 (uint16_t *data, uint16_t length, uint8_t init)
/* crc */
uint16_t crc16_table256_data16(uint16_t *data, uint16_t length, uint16_t init)
/* crc */
uint32_t crc32_table256_data16(uint16_t *data, uint16_t length, uint32_t init)
/* crc */

uint8_t  crc8_table16_data16 (uint16_t *data, uint16_t length, uint8_t init)
/* crc */
uint16_t crc16_table16_data16(uint16_t *data, uint16_t length, uint16_t init)
/* crc */
uint32_t crc32_table16_data16(uint16_t *data, uint16_t length, uint32_t init)
/* crc */

uint8_t  crc8_tabletwo16_data16 (uint16_t *data, uint16_t length, uint8_t
init) /* crc */
uint16_t crc16_tabletwo16_data16(uint16_t *data, uint16_t length, uint16_t
init) /* crc */
uint32_t crc32_tabletwo16_data16(uint16_t *data, uint16_t length, uint32_t
init) /* crc */
```

These functions calculate the CRC of *data* array of size *length* (16-bit words) using previous values of CRC *init*, using the respective CRC and table based implementation. The CRC function will be called on each byte of the 16-word, low-byte first then high byte, where the word is first retrieved from memory as a whole word.

See Section 11.3 for instructions to extend the library with CRC functions based on other polynomials. See Section 11.4 for instructions to extend the crc library with alternative CRC implementations (applied to reversed data, table based variants, etc.)

10. Trigonometric function approximations

The trigonometric functions are approximated with Taylor series and look-up tables.

Taylor series are used to approximate the value for the first octant ($0 \leq x < \pi/4$) and look-up table for the first octant or quadrant ($0 \leq x < \pi/2$), whereas the entire domain is covered using the various identities like: $\sin(-x) = -\sin(x)$, $\cos(x) = \sin(\pi/2 - x)$, ...

The approximation should be within 2LSBs from round (+/- 1LSB in case of floor) of exact floating point values.

The following Taylor series are available

$$\begin{aligned}\sin(x) &= x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots = x \left(1 - \frac{x^2}{2 \times 3} \left(1 - \frac{x^2}{4 \times 5} \left(1 - \frac{x^2}{6 \times 7} (1 - \dots) \right) \right) \right) \\ \cos(x) &= 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots = 1 - \frac{x^2}{2!} \left(1 - \frac{x^2}{3 \times 4} \left(1 - \frac{x^2}{5 \times 6} (1 - \dots) \right) \right) \\ \tan(x) &= x + \frac{x^3}{3} + \frac{2x^5}{15} + \frac{17x^7}{315} + \frac{62x^9}{2835} + \dots = x \left(1 + \frac{x^2}{3} + \frac{2x^4}{15} + \frac{17x^6}{315} + \frac{62x^8}{2835} + \dots \right)\end{aligned}$$

As (Taylor series for) $\tan(x)$ does not converge, the Taylor series approximation for $\tan(x)$ is only used for smaller values of x ($x < \pi/8$). For other values $\tan(x) = \sin(x) / \cos(x)$.

	MLX16x8		MLX16, MLX16-8	
	$0 < x < \pi/4$	$\pi/4 < x < \pi/2$	$0 < x < \pi/4$	$\pi/4 < x < \pi/2$
sin	Taylor 5th	$\cos(\pi/2-x)$	lookup table	lookup table
cos	Taylor 6th	$\sin(\pi/2-x)$	$\sin(\pi/2-x)$	$\sin(\pi/2-x)$
tan	Taylor 7th (till $\sim \pi/8$)	$\sin(x)/\cos(x)$	$\sin(x)/\cos(x)$	$\sin(x)/\cos(x)$
atan2	lookup table (y/x)	$\pi/2$ - lookup table (x/y)	lookup table (y/x)	$\pi/2$ - lookup table (x/y)

The Taylor series are implemented in assembler only for the MLX16x8 target CPU. C simulation versions are available. The lookup table versions are implemented in C and available for all targets. Use the define `__USE_TRIGONOMETRIC_LOOKUPTABLES__` to use the lookup table version for the MLX16x8 target CPU. (See also Section 11.2)

11. Library customization and configuration

The following utilities are necessary when extending the library as per Sections 11.3 etc.:

- m4 – macro preprocessor utility
- crc_tablegen – CRC table generation utility (provided with the math library)

11.1. C simulations

| Math library release 2.0 requires MLX16-GCC release 1.8 or later.

The math library functions can be simulated using internal C compiler routines. This feature can be helpful for the validation of the calculation results.

11.2. Trigonometric function approximations

The trigonometric function approximations using Taylor series are implemented in assembler only for the MLX16x8 target CPU. C simulation versions are available. (See Section 11.1).

The trigonometric function approximations using lookup table versions are implemented in C and available for all targets. These versions are the default for the MLX16 and MLX16-8 CPU targets; the Taylor series approximation is the default for the MLX16x8 CPU target

Use the define `__USE_TRIGONOMETRIC_LOOKUPTABLES__` to use the lookup table version for the MLX16x8 target CPU.

11.3. CRC functions with non-standard polynomials (libmath.a)

The crc8 and crc16 functions in the library use the standard polynomials:

- CRC8: $x^8 + x^7 + x^6 + x^4 + x^2 + 1$ (D5)
- CRC16 (CCITT): $x^{16} + x^{12} + x^5 + 1$ (1021)

To add crc functions with another polynomial add a (dummy) filename to `GEN_SRCS` variable in `libsrc/Makefile`:

- `crc16_nnnn.c` for a CRC16 variant using polynomial `nnnn`
- `crc8_nn.c` for a CRC8 variant using polynomial `nn`

This will result in the inclusion of the following functions in the library when the library is (re)built and installed:

```
uint8_t  crc8_nn(uint8_t x, uint8_t crc)
uint16_t crc16_nnnn (uint8_t x, uint16_t crc)
```

where *nnnn* is the polynomial specified.

For a CRC8 variant, the format of the polynomial should be a 2 digit hexadecimal value (without 0x prefix) that denotes the polynomial to use.

For a CRC16, the format of the polynomial should be a 4 digit hexadecimal value (without 0x prefix) that denotes the polynomial to use.

The value is the hexadecimal representation of the binary value that defines the polynomial. The top-bit of the polynomial should be omitted. For example:

$$x^8 + x^7 + x^6 + x^4 + x^2 + 1 \Rightarrow 1\ 1101\ 0101 \Rightarrow D5$$

11.4. CRC functions with non-standard polynomials (libcrc.a)

The `crc8`, `crc16` and `crc32` functions in the library use the standard polynomials:

- CRC8: $x^8 + x^7 + x^6 + x^4 + x^2 + 1$ (D5)
- CRC16 (CCITT): $x^{16} + x^{12} + x^5 + 1$ (1021)
- CRC32 (IEEE 802.3): $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ (04C11DB7)

11.4.1. Other polynomials

To add `crc` functions with another polynomial add a (dummy) filename to `GEN_SRCS` variable in `libsrc/crc/Makefile`:

- `genrcrc32_nnnnnnnn.c` for a CRC32 variant using polynomial *nnnnnnnn*
- `genrcrc16_nnnn.c` for a CRC16 variant using polynomial *nnnn*
- `genrcrc8_nn.c` for a CRC8 variant using polynomial *nn*

This will result in the inclusion of the following functions in the library when the library is (re)built and installed:

```
uint8_t  crc8_nn(uint8_t x, uint8_t crc)
uint16_t crc16_nnnn (uint8_t x, uint16_t crc)
uint32_t crc32_nnnnnnnn (uint8_t x, uint32_t crc)
```

where *nnnn* is the polynomial specified.

For a CRC8 variant, the format of the polynomial should be a 2 digit hexadecimal value (without 0x prefix) that denotes the polynomial to use.

For a CRC16, the format of the polynomial should be a 4 digit hexadecimal value (without 0x prefix) that denotes the polynomial to use.

For a CRC32, the format of the polynomial should be a 8 digit hexadecimal value (without 0x prefix) that denotes the polynomial to use.

The value is the hexadecimal representation of the binary value that defines the polynomial. The top-bit of the polynomial should be omitted. For example:

$$x^8 + x^7 + x^6 + x^4 + x^2 + 1 \Rightarrow 1\ 1101\ 0101 \Rightarrow D5$$

11.4.2. Reflected variants

To add `crc` functions that are applied on reversed data add a (dummy) filename to `GEN_SRCS` variable in `libsrc/crc/Makefile`:

- *gencrc32r_nnnnnnnn.c* for a CRC32 variant using polynomial *nnnnnnnn* on reflected/reversed data byte
- *gencrc16r_nnnn.c* for a CRC16 variant using polynomial *nnnn* on reflected/reversed data byte
- *gencrc8r_nn.c* for a CRC8 variant using polynomial *nn* on reflected/reversed data byte

This will result in the inclusion of the following functions in the library when the library is (re)built and installed:

```
uint8_t  crc8r_nn(uint8_t x, uint8_t crc)
uint16_t crc16r_nnnn (uint8_t x, uint16_t crc)
uint32_t crc32r_nnnnnnnn (uint8_t x, uint32_t crc)
```

where *nnnn* is the polynomial specified. If *nnnn* is not specified, the default polynomial is used.

```
uint16_t crc16r(uint8_t c, uint16_t crc)
{
    uint16_t i;
    crc = crc ^ (uint16_t) c;
    for (i = 0; i < 8; ++i)
    {
        if ((crc & 1) != 0)
            crc = (crc >> 1) ^ 0x8408;
        else
            crc = crc >> 1;
    }
    return crc;
}
```

C-code 27

11.4.3. 256 entry table based implementation

To add crc functions with a table-based CRC implementation using a table of 256 entries add a (dummy) filename to *GEN_SRCS* variable in *libsrc/crc/Makefile*:

- *gencrc32t256_nnnnnnnn.c* for a CRC32 variant using polynomial *nnnnnnnn*
- *gencrc16t256_nnnn.c* for a CRC16 variant using polynomial *nnnn*
- *gencrc8t256_nn.c* for a CRC8 variant using polynomial *nn*

This will result in the inclusion of the following functions in the library when the library is (re)built and installed:

```
uint8_t  crc8_nn_table256(uint8_t x, uint8_t crc)
uint16_t crc16_nnnn_table256(uint8_t x, uint16_t crc)
uint32_t crc32_nnnnnnnn_table256 (uint8_t x, uint32_t crc)

uint8_t  crc8_nn_table256_data8(uint8_t *data, uint16_t length, uint8_t crc)
uint16_t crc16_nnnn_table256_data8(uint8_t *data, uint16_t length, uint16_t
crc)
uint32_t crc32_nnnnnnnn_table256_data8(uint8_t *data, uint16_t length,
uint32_t crc)

uint8_t  crc8_nn_table256_data16(uint16_t *data, uint16_t length, uint8_t crc)
uint16_t crc16_nnnn_table256_data16(uint16_t *data, uint16_t length, uint16_t
crc)
uint32_t crc32_nnnnnnnn_table256_data16(uint16_t *data, uint16_t length,
uint32_t crc)
```

where *nnnn* is the polynomial specified. If *nnnn* is not specified, the default polynomial is used.

Note that the respective CRC tables of 256 entries (in ROM) will also be generated.

To add crc functions with a table-based CRC implementation using a table of 256 entries and that are applied to reversed data add a (dummy) filename to *GEN_SRCS* variable in *libsrc/crc/Makefile*:

- *gencrc32rt256_nnnnnnnn.c* for a CRC32 variant using polynomial *nnnnnnnn* on reflected/reversed data byte
- *gencrc16rt256_nnnn.c* for a CRC16 variant using polynomial *nnnn* on reflected/reversed data byte
- *gencrc8rt256_nn.c* for a CRC8 variant using polynomial *nn* on reflected/reversed data byte

This will result in the inclusion of the following functions in the library when the library is (re)built and installed:

```
uint8_t  crc8r_nn_table256(uint8_t x, uint8_t crc)
uint16_t crc16r_nnnn_table256(uint8_t x, uint16_t crc)
uint32_t crc32r_nnnnnnnn_table256 (uint8_t x, uint32_t crc)

uint8_t  crc8r_nn_table256_data8(uint8_t *data, uint16_t length, uint8_t crc)
uint16_t crc16r_nnnn_table256_data8(uint8_t *data, uint16_t length, uint16_t
crc)
uint32_t crc32r_nnnnnnnn_table256_data8(uint8_t *data, uint16_t length,
uint32_t crc)

uint8_t  crc8r_nn_table256_data16(uint16_t *data, uint16_t length, uint8_t
crc)
uint16_t crc16r_nnnn_table256_data16(uint16_t *data, uint16_t length, uint16_t
crc)
uint32_t crc32r_nnnnnnnn_table256_data16(uint16_t *data, uint16_t length,
uint32_t crc)
```

where *nnnn* is the polynomial specified. If *nnnn* is not specified, the default polynomial is used.

Note that the respective CRC tables of 256 entries (in ROM) will also be generated.

- *gencrc32t256_nnnnnnnn.c* for a CRC32 variant using polynomial *nnnnnnnn*
- *gencrc16t256_nnnn.c* for a CRC16 variant using polynomial *nnnn*
- *gencrc8t256_nn.c* for a CRC8 variant using polynomial *nn*

11.4.4. 16 entry table based implementation

To add crc functions with a table-based CRC implementation using a (single) table of 16 entries add a (dummy) filename to *GEN_SRCS* variable in *libsrc/crc/Makefile*:

- *gencrc32t16_nnnnnnnn.c* for a CRC32 variant using polynomial *nnnnnnnn*
- *gencrc16t16_nnnn.c* for a CRC16 variant using polynomial *nnnn*
- *gencrc8t16_nn.c* for a CRC8 variant using polynomial *nn*

This will result in the inclusion of the following functions in the library when the library is (re)built and installed:

```
uint8_t  crc8_nn_table16(uint8_t x, uint8_t crc)
uint16_t crc16_nnnn_table16(uint8_t x, uint16_t crc)
uint32_t crc32_nnnnnnnn_table16 (uint8_t x, uint32_t crc)
```



```
uint8_t  crc8_nn_table16_data8(uint8_t *data, uint16_t length, uint8_t crc)
uint16_t crc16_nnnn_table16_data8(uint8_t *data, uint16_t length, uint16_t
crc)
uint32_t crc32_nnnnnnnn_table16_data8(uint8_t *data, uint16_t length, uint32_t
crc)

uint8_t  crc8_nn_table16_data16(uint16_t *data, uint16_t length, uint8_t crc)
uint16_t crc16_nnnn_table16_data16(uint16_t *data, uint16_t length, uint16_t
crc)
uint32_t crc32_nnnnnnnn_table16_data16(uint16_t *data, uint16_t length,
uint32_t crc)
```

where *nnnn* is the polynomial specified. If *nnnn* is not specified, the default polynomial is used.

Note that the respective CRC tables of 16 entries (in ROM) will also be generated.

To add crc functions with a table-based CRC implementation using a (single) table of 16 entries and that are applied to reversed data add a (dummy) filename to *GEN_SRCS* variable in *libsrc/crc/Makefile*:

- *gencrc32rt16_nnnnnnnn.c* for a CRC32 variant using polynomial *nnnnnnnn* on reflected/reversed data byte
- *gencrc16rt16_nnnn.c* for a CRC16 variant using polynomial *nnnn* on reflected/reversed data byte
- *gencrc8rt16_nn.c* for a CRC8 variant using polynomial *nn* on reflected/reversed data byte

This will result in the inclusion of the following functions in the library when the library is (re)built and installed:

```
uint8_t  crc8r_nn_table16 (uint8_t x, uint8_t crc)
uint16_t crc16r_nnnn_table16(uint8_t x, uint16_t crc)
uint32_t crc32r_nnnnnnnn_table16 (uint8_t x, uint32_t crc)

uint8_t  crc8r_nn_table16_data8(uint8_t *data, uint16_t length, uint8_t crc)
uint16_t crc16r_nnnn_table16_data8(uint8_t *data, uint16_t length, uint16_t
crc)
uint32_t crc32r_nnnnnnnn_table16_data8(uint8_t *data, uint16_t length,
uint32_t crc)

uint8_t  crc8r_nn_table16_data16(uint16_t *data, uint16_t length, uint8_t crc)
uint16_t crc16r_nnnn_table16_data16(uint16_t *data, uint16_t length, uint16_t
crc)
uint32_t crc32r_nnnnnnnn_table16_data16(uint16_t *data, uint16_t length,
uint32_t crc)
```

where *nnnn* is the polynomial specified. If *nnnn* is not specified, the default polynomial is used.

Note that the respective CRC tables of 16 entries (in ROM) will also be generated.

11.4.5. Two 16 entry table based implementation

To add crc functions with a table-based CRC implementation using two tables of 16 entries add a (dummy) filename to *GEN_SRCS* variable in *libsrc/crc/Makefile*:

- *gencrc32two16_nnnnnnnn.c* for a CRC32 variant using polynomial *nnnnnnnn*
- *gencrc16two16_nnnn.c* for a CRC16 variant using polynomial *nnnn*
- *gencrc8two16_nn.c* for a CRC8 variant using polynomial *nn*

This will result in the inclusion of the following functions in the library when the library is (re)built and installed:

```
uint8_t  crc8_nn_tabletwo16(uint8_t x, uint8_t crc)
```

```
uint16_t crc16_nnnn_tabletwo16(uint8_t x, uint16_t crc)
uint32_t crc32_nnnnnnnn_tabletwo16 (uint8_t x, uint32_t crc)

uint8_t  crc8_nn_table16_datatwo8(uint8_t *data, uint16_t length, uint8_t crc)
uint16_t crc16_nnnn_table16_datatwo8(uint8_t *data, uint16_t length, uint16_t
crc)
uint32_t crc32_nnnnnnnn_table16_datatwo8(uint8_t *data, uint16_t length,
uint32_t crc)

uint8_t  crc8_nn_tabletwo16_data16(uint16_t *data, uint16_t length, uint8_t
crc)
uint16_t crc16_nnnn_tabletwo16_data16(uint16_t *data, uint16_t length,
uint16_t crc)
uint32_t crc32_nnnnnnnn_tabletwo16_data16(uint16_t *data, uint16_t length,
uint32_t crc)
```

where *nnnn* is the polynomial specified. If *nnnn* is not specified, the default polynomial is used.

Note that the respective CRC tables of 16 entries (in ROM) will also be generated.

To add crc functions with a table-based CRC implementation using two tables of 16 entries and that are applied to reversed data add a (dummy) filename to *GEN_SRCS* variable in *libsrc/crc/Makefile*:

- *gencrc32rt16_nnnnnnnn.c* for a CRC32 variant using polynomial *nnnnnnnn* on reflected/reversed data byte
- *gencrc16rt16_nnnn.c* for a CRC16 variant using polynomial *nnnn* on reflected/reversed data byte
- *gencrc8rt16_nn.c* for a CRC8 variant using polynomial *nn* on reflected/reversed data byte

This will result in the inclusion of the following functions in the library when the library is (re)built and installed:

```
uint8_t  crc8r_nn_tabletwo16 (uint8_t x, uint8_t crc)
uint16_t crc16r_nnnn_tabletwo16(uint8_t x, uint16_t crc)
uint32_t crc32r_nnnnnnnn_tabletwo16 (uint8_t x, uint32_t crc)

uint8_t  crc8r_nn_tabletwo16_data8(uint8_t *data, uint16_t length, uint8_t
crc)
uint16_t crc16r_nnnn_tabletwo16_data8(uint8_t *data, uint16_t length, uint16_t
crc)
uint32_t crc32r_nnnnnnnn_tabletwo16_data8(uint8_t *data, uint16_t length,
uint32_t crc)

uint8_t  crc8r_nn_tabletwo16_data16(uint16_t *data, uint16_t length, uint8_t
crc)
uint16_t crc16r_nnnn_tabletwo16_data16(uint16_t *data, uint16_t length,
uint16_t crc)
uint32_t crc32r_nnnnnnnn_tabletwo16_data16(uint16_t *data, uint16_t length,
uint32_t crc)
```

where *nnnn* is the polynomial specified. If *nnnn* is not specified, the default polynomial is used.

Note that the respective CRC tables of 16 entries (in ROM) will also be generated.

11.5. (Low pass) Filter functions with other coefficients (*libfilter.a*)

To add first order low pass filter functions with coefficient *m* ($1 \leq n < 16$), add a (dummy) filename to *GEN_SRCS* variable in *libsrc/filter/Makefile*:

- *genlpfn_m.c* for a first order low pass filter with coefficient *m* supporting half the input range of a 16-bit unsigned integer
- *genlpfnU16_m.c* for a first order low pass filter with coefficient *m* supporting the entire input range of a 16-bit unsigned integer
- *genlpfnI16_m.c* for a first order low pass filter with coefficient *m* supporting the entire input range of a 16-bit signed integer

This will result in the inclusion of the following functions in the library when the library is (re)built and installed:

```
uint16_t lpfm      (uint16_t in, uint16_t *p_out)
uint16_t lpfm_U16(uint16_t in, uint16_t *p_out)
int16_t  lpfm_I16(int16_t in, int16_t *p_out)
```

where *m* is the coefficient.